



MACHINE LEARNING CON PYTHON

TEMA 2. ENTORNO DE
DESARROLLO

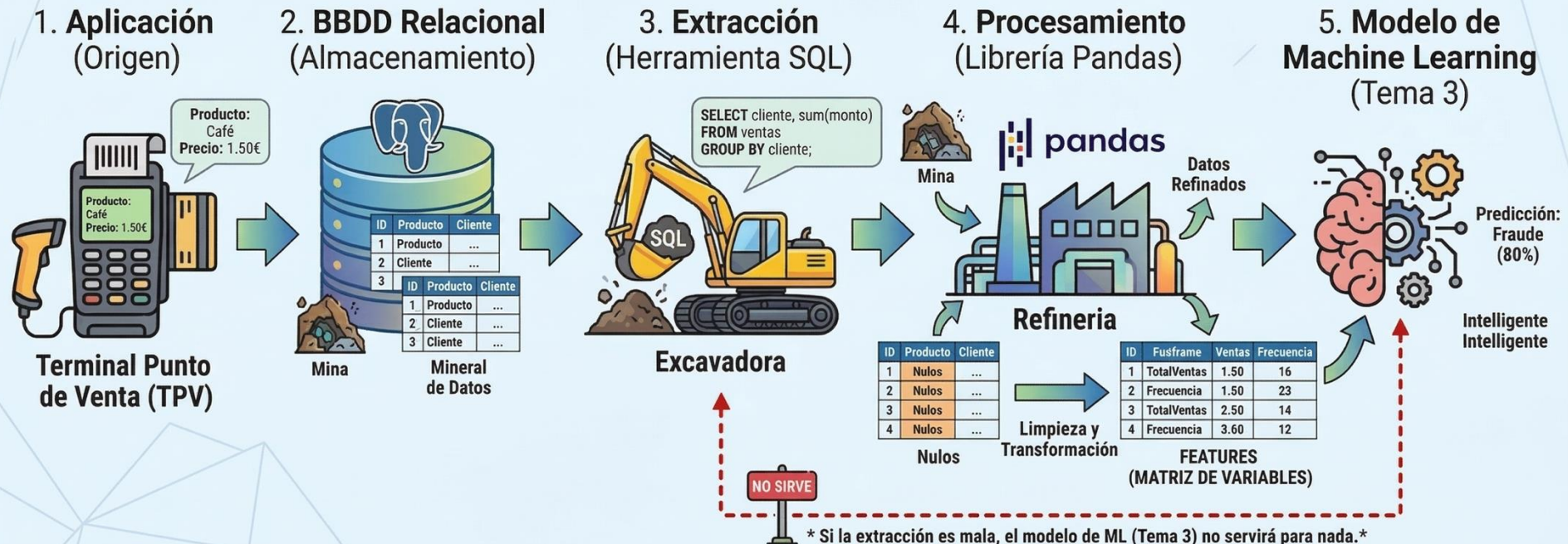


Contenidos Tema 2

- / 1. Fundamentos de BBDD Relacionales**
- / 2. Acceso y consulta a BBDD relacionales con SQL desde Python**
- / 3. Nociones de BBDD no Relacionales**
- / 4. Servicios Web**
- / 5. Archivos CSV (Comma Separated Values)**
- / 6. IDE (Visual Studio Code) vs. Libretas (Google Colab)**

EL VIAJE DEL DATO: DE LA CAPTURA AL MODELO

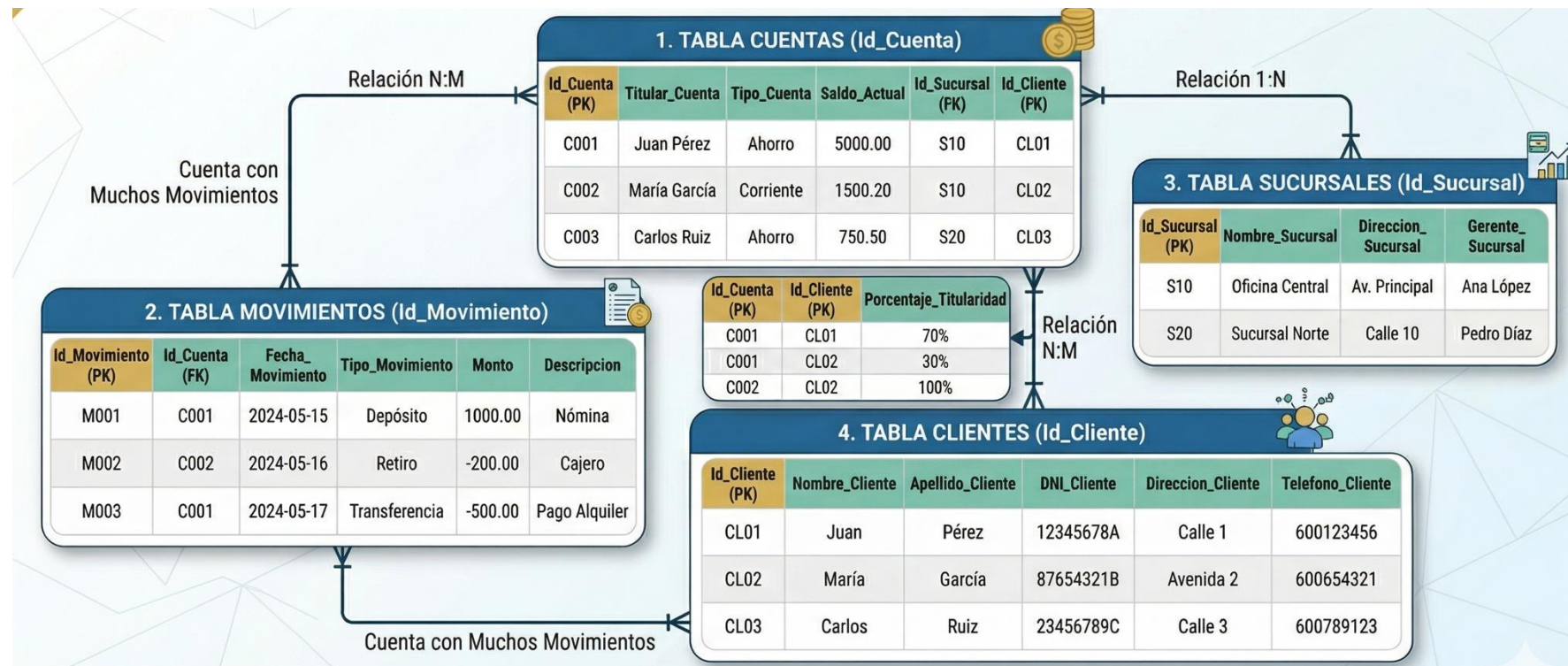
'El viaje del dato: De la transacción al modelo.'



1. FUNDAMENTOS DE BBDD RELACIONALES

¿QUÉ ES UNA BBDD RELACIONAL?

- Estructura tabular estricta (filas y columnas)
- Modelo **Entidad-Relación**: Entidades (tablas) y relaciones entre éstas
- Garantiza transaccionalidad → Propiedades **ACID**: **Atomicidad, Consistencia, Aislamiento, Durabilidad**
- Estándares de la industria: *PostgreSQL, MySQL, SQLite*.



ANATOMÍA DE UNA TABLA (EJEMPLO TABLA "CLIENTES_RIESGO")

- Cada fila (tupla) es una observación, un caso específico de la entidad que simboliza la table, p.ej. cliente
- Cada columna es un atributo → En ML, algunos de estos atributos pasarán a ser *features*.

ID_Cliente	Empresa	Sector	Score_Crediticio
101	TechNova SL	Tecnología	0.85
102	RetailSur SA	Comercio	0.42
103	BioFarma	Salud	0.91

- A nivel estructural, una tabla es prácticamente análoga a un DataFrame de Pandas (tema 4).

1. FUNDAMENTOS DE BBDD RELACIONALES

CLAVES PRIMARIAS (PK) Y CLAVES FORÁNEAS (FK)

El nexo de unión de los datos en BBDDs

- **Clave primaria** (primary key, PK): identificador único de cada tupla en una tabla → ID_Cliente

ID_Cliente	Empresa	Sector	Score_Crediticio
101	TechNova SL	Tecnología	0.85
102	RetailSur SA	Comercio	0.42
103	BioFarma	Salud	0.91

- **Clave foránea** (foreign key, FK): columna(s) de una tabla X que actuá(n) como PK de otra tabla Y

ID_Operacion (PK)	ID_Cliente (FK)	Importe (€)
A-001	101	50,000
A-002	101	15,000
B-001	102	120,000

TIPOS DE DATOS EN TABLAS RELACIONALES Y SU IMPACTO EN ML

Concepto de Negocio	Ejemplo	Tipo en BBDD (SQL)	Tipo en Python (Pandas)
Divisas / SalDOS	15000.50	DECIMAL(10,2)	float64
Fechas de Vencimiento	2024-12-31	DATE	datetime64
Categorías (Sectores)	'Tecnología'	VARCHAR(50)	object / category

- Hay que tener cuidado y revisar siempre bajo qué tipo están guardados los datos de una columna en una BBDD → al extraerlos y pasar al "territorio de Python", a veces podría ser necesario realizar conversiones entre datos.

INTEGRIDAD REFERENCIAL

Salvaguarda del modelo relacional: si intentásemos borrar al cliente 101, sus dos operaciones de préstamo en la tabla relacionada quedarían "huérfanos"

DELETE FROM CLIENTES
WHERE ID_CLIENTE = 101;



**ERROR: foreign key
constraint violation**

ID_Cliente	Empresa	Sector	Score_Crediticio
101	TechNova SL	Tecnología	0.85
102	RetailSur SA	Comercio	0.42
103	BioFarma	Salud	0.91

ID_Operacion (PK)	ID_Cliente (FK)	Importe (€)
A-001	101	50,000
A-002	101	15,000
B-001	102	120,000

SQL: El lenguaje universal de los datos

Structured Query Language

DDL (Definición): CREATE, DROP → Crear contenedores (Perfil IT).

DML/DQL (Manipulación/Consulta): SELECT, UPDATE → Extraer y actualizar información (Perfil Analista/Data Scientist).



Como equipo financiero o de Data Scientists, vuestra herramienta de trabajo diaria con BBDDs relacionales es el **SELECT**: el objetivo es extraer (consultar) datos para entrenar modelos, no administrar servidores

FILTRADO BÁSICO DE DATOS PARA MACHINE LEARNING

- **SELECT ... FROM:** qué columnas escoger, y de qué tabla (usar '*' para filtrar todas las columnas).
- **WHERE:** qué filas escoger / qué condición deben cumplir

ID_Cliente	Empresa	Sector	Score_Crediticio
101	TechNova SL	Tecnología	0.85
102	RetailSur SA	Comercio	0.42
103	BioFarma	Salud	0.91

CLIENTES

ID_Operacion (PK)	ID_Cliente (FK)	Importe (€)
A-001	101	50,000
A-002	101	15,000
B-001	102	120,000

PRESTAMOS

SELECT ID_Cliente, Importe **FROM** PRESTAMOS **WHERE** Importe > 20000;

ID_Cliente	Importe
101	50,000
102	120,000

AGREGACIONES: HACIENDO *Feature Engineering* EN ORIGEN

- **GROUP BY:** permite resumir información

ID_Operacion (PK)	ID_Cliente (FK)	Importe (€)
A-001	101	50,000
A-002	101	15,000
B-001	102	120,000

PRESTAMOS

SELECT ID_Cliente, **SUM**(Importe) as Deuda_Total

FROM PRESTAMOS **GROUP BY** ID_Cliente;

ID_Cliente	Deuda_Total
101	65,000
102	120,000

CRUCE DE INFORMACIÓN ENTRE TABLAS (JOIN)

- Útil cuando vamos a construir un dataset a partir de datos provenientes de varias tablas

ID_Cliente	Empresa	Sector	Score_Crediticio
101	TechNova SL	Tecnología	0.85
102	RetailSur SA	Comercio	0.42
103	BioFarma	Salud	0.91

CLIENTES

ID_Operacion (PK)	ID_Cliente (FK)	Importe (€)
A-001	101	50,000
A-002	101	15,000
B-001	102	120,000

PRESTAMOS

SELECT C.Empresa, P.Importe

FROM CLIENTES C

INNER JOIN PRESTAMOS P **ON** C.ID_Cliente = P.ID_Cliente;

Empresa	Importe
TechNova SL	50,000
TechNova SL	15,000
RetailSur SA	120,000

REGLA DE ORO: BBDD vs RAM

Buenas prácticas

- **BBDD (SQL):** Servidores potentes. Ideal para: Filtrar (WHERE), Unir (JOIN), Agregar (GROUP BY).
- **Python (Pandas/RAM):** Memoria local. Ideal para: Limpiar Valores Nulos, Transformar datos estadísticamente, Entrenar Modelos.



Las bases de datos están diseñadas para buscar. Tu equipo local no. Si intentas cargar un histórico de 10 años de transacciones en Pandas para luego filtrar solo el año 2023, la memoria de tu máquina podría colapsar. Filtra primero mediante SQL, analiza después en Python."

INTEGRACIÓN DE BBDDs + PYTHON

- **SQLite:** Viene integrado en Python (*import sqlite3*). Sin servidores. Tu BBDD es un simple archivo .db. Ideal para aprender hoy.
- **PostgreSQL / MySQL:** Código abierto, robustas. Necesitan servidor. Ideales para producción corporativa.

```
import sqlite3
# 1. Conectar al archivo (nuestra BBDD)
conexion = sqlite3.connect('cartera_financiera.db')
cursor = conexion.cursor()

# 2. Extraer datos en crudo
cursor.execute("SELECT Empresa FROM Clientes LIMIT 2")
print(cursor.fetchall())

conexion.close()
```

DE CONSULTA SQL A *DataFrame* DE PANDAS

- **La llave maestra:** el método `pd.read_sql_query()`

```
import pandas as pd
import sqlite3

# 1. Conectamos al archivo que acabamos de generar
conn = sqlite3.connect('cartera_financiera.db')

# 2. Consultamos la tabla real 'Clientes' buscando perfiles de riesgo
(Score bajo)
query = "SELECT Empresa, Sector, Score_Crediticio FROM Clientes WHERE
Score_Crediticio < 0.7"

# 3. ¡Magia! Directamente a un DataFrame listo para Machine Learning
df_riesgo = pd.read_sql_query(query, conn)
print(df_riesgo.head())
```

	Empresa	Sector	Score_Crediticio
0	RetailSur SA	Comercio	0.42
1	Construcciones Iberia	Inmobiliario	0.65
2	AgroSur	Agricultura	0.55

ORM: Preparando el modelo de datos para el mundo real

¿Qué es un ORM (*Object-Relational Mapping*)?

- Un mecanismo que convierte las tablas de la base de datos en Clases normales de Python.
- La herramienta "líder" para ORM es la librería **SQLAlchemy**.
- En el contexto de ML, ORM es un valioso aliado para hacer posible un MLOps eficaz
- **Para extraer datos masivos (Pasado):** bastaría con usar SQL puro + Pandas (read_sql).
- **Para guardar predicciones individuales (Futuro):** conviene usar ORM.

```
INSERT INTO Prestamos (ID_Operacion, ID_Cliente, Importe) VALUES ('G-001', 101, 45000);
```



```
nuevo_prestamo = Prestamo(id_operacion='G-001', cliente=101, importe=45000)
session.add(nuevo_prestamo)
session.commit()
```

Documentación SQLAlchemy: <https://docs.sqlalchemy.org/en/20/>

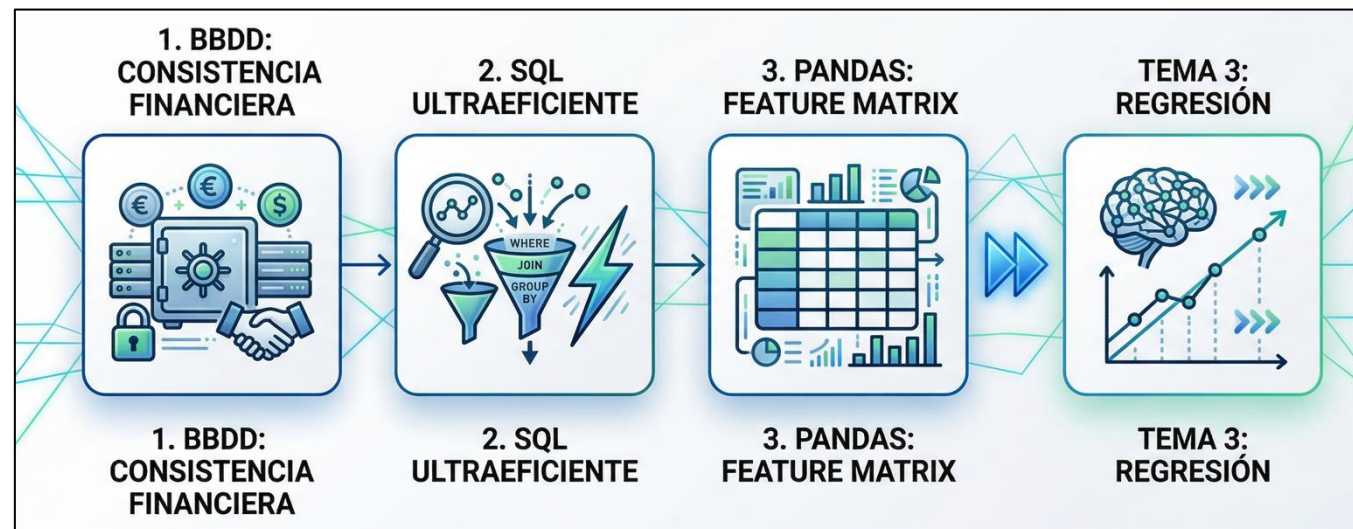
1. FUNDAMENTOS DE BBDD RELACIONALES

RESUMEN

- Las bases de datos relacionales aportan consistencia a negocios en sectores críticos como el financiero
- SQL nos permite **filtrar (WHERE)**, **unir (JOIN)** y **agregar (GROUP BY)** de forma muy eficiente.
- Pandas captura esa información y la convierte en un *dataset* consistente en *features*

▶▶ Próximo paso (Tema 3):

Entrenar modelos de ML sencillos (Regresión) con DataFrames.



MÁS ALLÁ DEL SQL CONVENCIONAL

NoSQL es un paradigma distinto al SQL tradicional, que ofrece **flexibilidad** para trabajar con **datos no totalmente estructurados** → contratos en PDF, logs de servidores, feeds de social media, etc.

¿Qué es?: Sistemas que no usan el esquema rígido de filas/columnas.

¿Cuándo se usa?:

- Ante un volumen masivo de datos (Big Data).
- Datos que cambian constantemente de forma.
- Se requiere rapidez extrema de lectura/escritura.

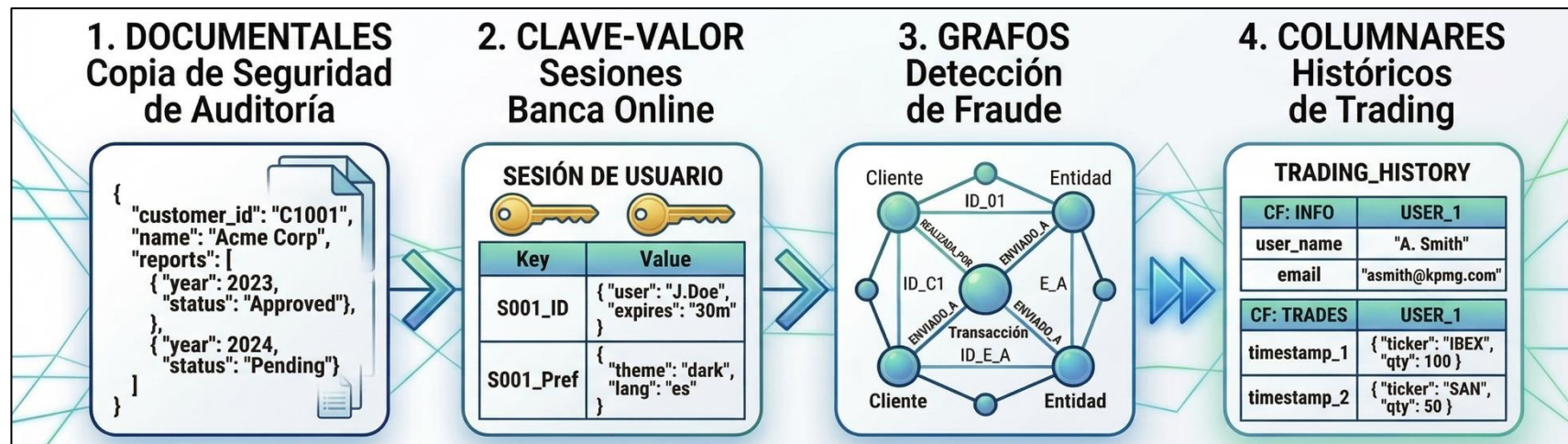
Concepto clave: "*Schema-on-read*" (se brinda estructura al leer el dato, no al guardarlo).

OJO AL "DATO": NoSQL no sustituye a las BBDD relacionales, las complementa donde el SQL pueda resultar rígido.

3. NOCIONES DE BBDD NO RELACIONALES

TIPOS DE ENFOQUES NoSQL Y CASOS DE USO

Tipo	Concepto	Ejemplo Financiero	Tecnología Líder
Documentales	Guardan JSONs (como fichas)	Expedientes de Auditoría	MongoDB
Clave-Valor	Diccionarios ultra-rápidos	Sesiones de banca online	Redis
Grafos	Foco en las relaciones	Detección de Fraude	Neo4j
Columnares	Lectura masiva de columnas	Históricos de Trading	Cassandra



SQL vs NoSQL: ¿CUÁL ELEGIR?

Elige SQL (Relacional) si...

- Necesitas integridad total (ACID), *reporting* financiero estándar y transacciones bancarias complejas.

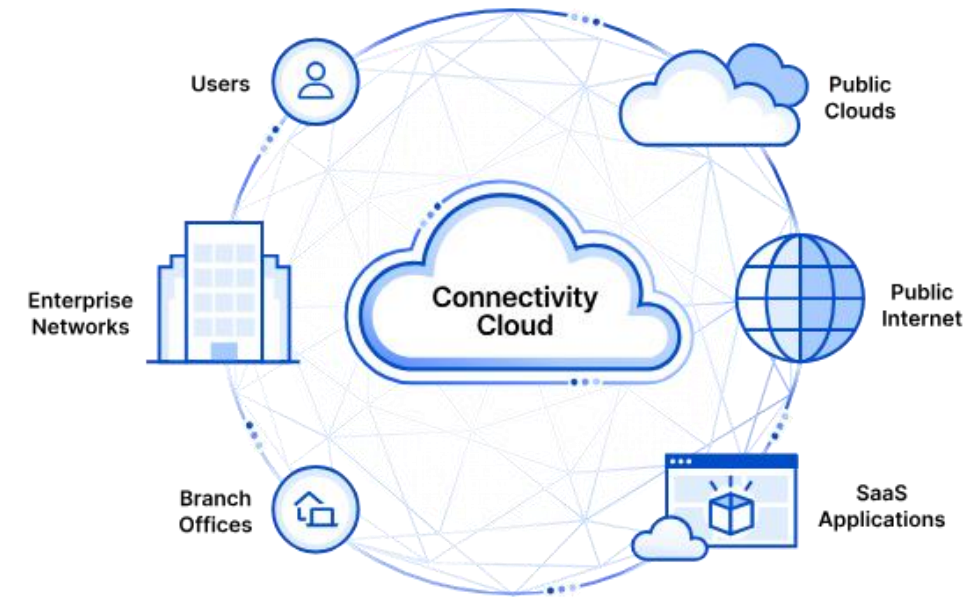
Elige NoSQL si...

- Tienes datos masivos desestructurados, necesitas escalar horizontalmente de forma barata o buscas analizar relaciones complejas (redes).

En Machine Learning, no es nada raro extraer de BBDDs NoSQL para terminar creando una tabla más "SQL-friendly" a modo de Dataframe en Pandas.

EL PUENTE ENTRE IA Y USUARIO FINAL

- Un servicio web (API) permite que tu modelo de Machine Learning se conecte con aplicaciones reales.
- Permite que una app móvil o sitio web envíe datos mediante una "petición" y reciba una predicción.
- Es la forma en la que aplicaciones como ChatGPT o Netflix nos ofrecen resultados.
- Actúan como un traductor universal entre sistemas.



En una etapa avanzada del curso, experimentaremos con Demos para el despliegue de modelos de machine learning entrenados por nosotros como un servicio Web

SIMPLICIDAD ESCALABLE: DATASETS PORTABLES

El **formato CSV** (*Comma Separated Values*) es la piedra angular del intercambio de datos en ciencia de datos.

- **Ligero:** No tiene adornos, solo datos puros.
- **Compatible:** Se lee igual de bien en Excel que en Python.
- **Estructurado:** Filas y columnas que cualquier algoritmo puede entender.

EN ESTE REPOSITORIO, DE ACCESO PÚBLICO Y GRATUITO, PUEDES VER DE PRIMERA MANO VARIOS DATASETS EN FORMATO CSV:

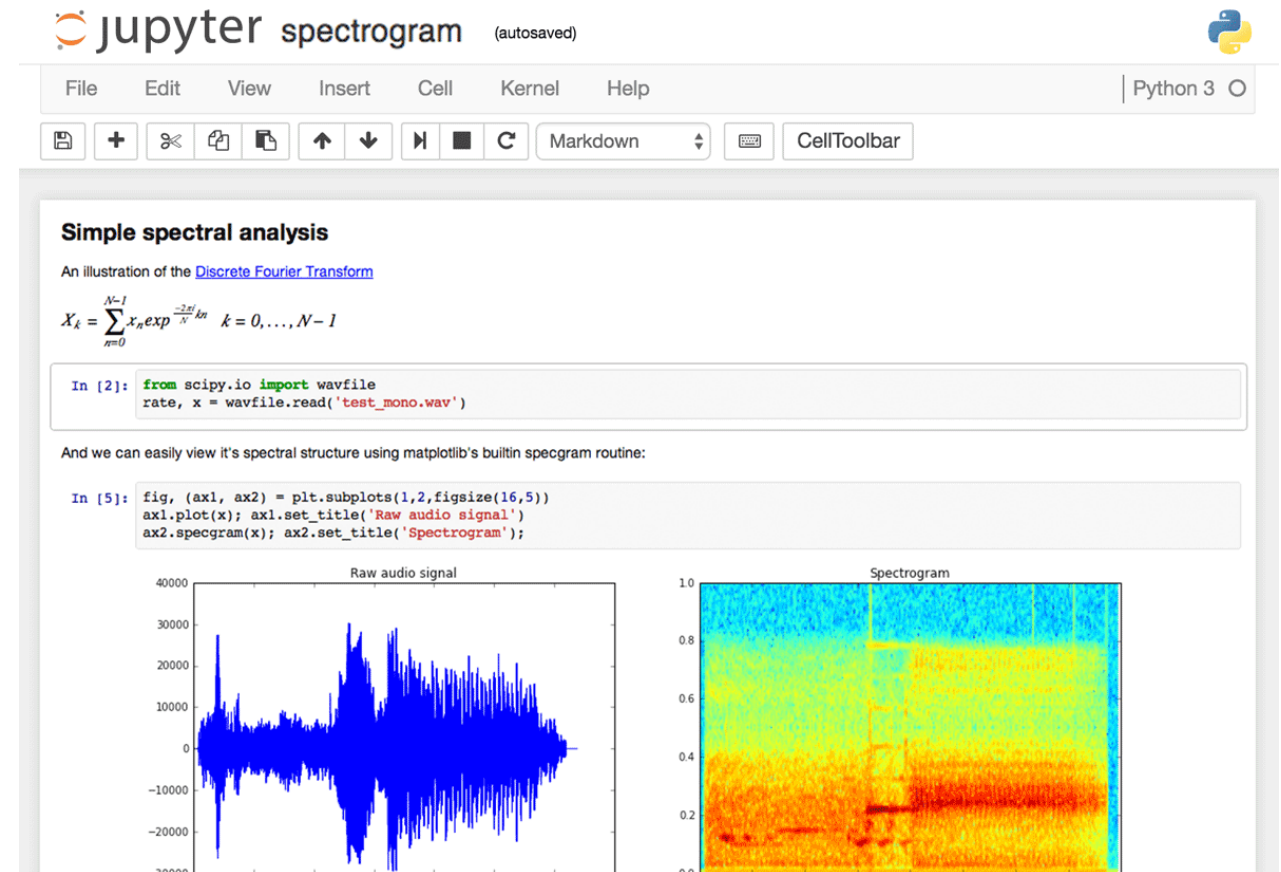
<https://github.com/gakudo-ai/open-datasets>



Libretas: El Laboratorio de creación y evaluación de modelos

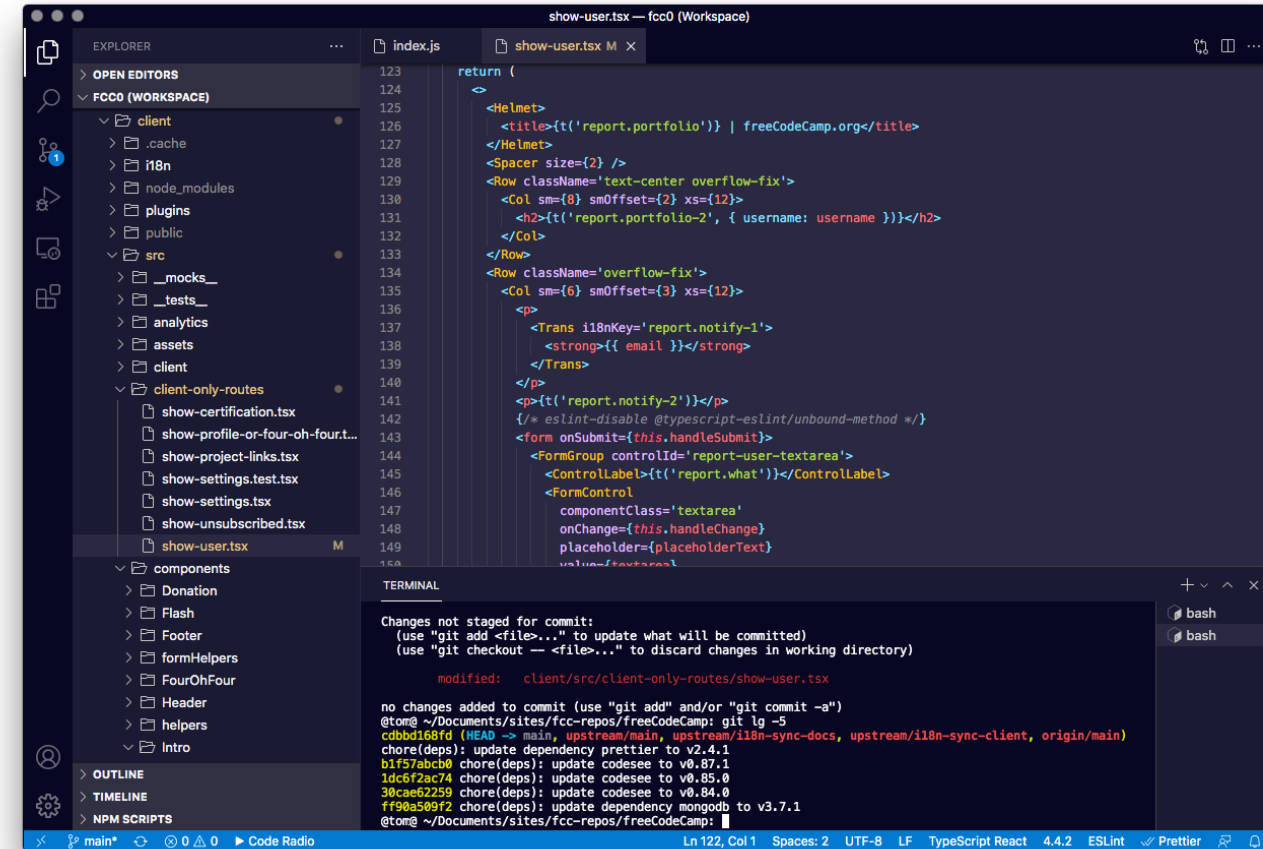
- Exploración y Rapidez
- Herramientas como Jupyter o Google Colab son ideales para las fases iniciales del Machine Learning, antes del despliegue de modelos a producción.
- Ejecutas código paso a paso.
- Visualizas gráficos y resultados al instante.
- Perfecto para aprender, documentar y experimentar con datos.

EN EL CURSO, UTILIZAREMOS
PRINCIPALMENTE GOOGLE COLAB



IDEs: La Factoría de modelos de IA

- Software Profesional
- Es en entornos integrados como Visual Studio Codem Anaconda, o PyCharm donde los modelos se convierten en productos robustos.
- Diseñados para manejar miles de líneas de código.
- Herramientas avanzadas de búsqueda y depuración de errores.
- Esencial para trabajar en equipo y automatizar procesos.



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the file structure of a project named 'fcc0 (Workspace)'. The file 'show-user.tsx' is selected under the 'client-only-routes' directory. The main editor area shows the content of 'show-user.tsx', which is a React component using JSX. The code includes a return statement with a JSX element containing a form for user registration. The form has a text input for 'report.what' and a submit button. The terminal window at the bottom shows the output of a 'git log' command, displaying commit history with dates and commit hashes.

```
return (  
  <Helmet>  
    <title>{t('report.portfolio')} | freeCodeCamp.org</title>  
  </Helmet>  
  <Spacer size={2} />  
  <Row className='text-center overflow-fix'>  
    <Col sm={8} smOffset={2} xs={12}>  
      <h2>{t('report.portfolio-2', { username: username })}</h2>  
    </Col>  
  </Row>  
  <Row className='overflow-fix'>  
    <Col sm={6} smOffset={3} xs={12}>  
      <p>  
        <Trans i18nKey='report.notify-1'>  
          <strong>{{ email }}</strong>  
        </Trans>  
      </p>  
      <p>{t('report.notify-2')}</p>  
      </* eslint-disable @typescript-eslint/unbound-method */>  
      <form onSubmit={this.handleSubmit}>  
        <FormGroup controlId='report-user-textarea'>  
          <ControlLabel>{t('report.what')}</ControlLabel>  
          <FormControl  
            className='textarea'  
            onChange={this.handleChange}  
            placeholder={placeholderText}  
            value={textarea} />  
      </form>  
    </Col>  
  </Row>  
)
```

Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git checkout -- <file>..." to discard changes in working directory)

modified: client/src/client-only-routes/show-user.tsx

no changes added to commit (use "git add" and/or "git commit -a")
@tom0 ~/Documents/sites/fcc-repos/freeCodeCamp: git lg -5
cbbbd168fd (HEAD -> main, upstream/main, upstream/i18n-sync-docs, upstream/i18n-sync-client, origin/main)
chore(deps): update dependency prettier to v2.4.1
b1f57abcb0 chore(deps): update codesee to v0.87.1
1dc6f2ac74 chore(deps): update codesee to v0.85.0
30cae62259 chore(deps): update codesee to v0.84.0
ff90a509f2 chore(deps): update dependency mongodb to v3.7.1
@tom0 ~/Documents/sites/fcc-repos/freeCodeCamp: